Enhancing Certifiable Robustness via a Deep Model Ensemble

Huan Zhang
UCLA
huan@huan-zhang.com

Minhao Cheng
UCLA
mhcheng@ucla.edu

Cho-Jui Hsieh
UCLA
chohsieh@cs.ucla.edu

arXiv:1910.14655v1 [stat.ML] 31 Oct 2019

Abstract

We propose an algorithm to enhance certified robustness of a deep model ensemble by optimally weighting each base model. Unlike previous works on using ensembles to empirically improve robustness, our algorithm is based on optimizing a guaranteed robustness certificate of neural networks. Our proposed ensemble framework with certified robustness, **RobBoost**, formulates the optimal model selection and weighting task as an optimization problem on a lower bound of classification margin, which can be efficiently solved using coordinate descent. Experiments show that our algorithm can form a more robust ensemble than naively averaging all available models using robustly trained MNIST or CIFAR base models. Additionally, our ensemble typically has better accuracy on clean (unperturbed) data. RobBoost allows us to further improve certified robustness and clean accuracy by creating an ensemble of already certified models.

1 Introduction

The lack of robustness in deep neural networks (DNNs) has motivated recent research on verifying and improving the robustness of DNN models (Katz et al., 2017; Dvijotham et al., 2018c; Gehr et al., 2018; Singh et al., 2018; Madry et al., 2018; Raghunathan et al., 2018b; Wong and Kolter, 2018; Zhang et al., 2018). Improving the robustness of neural networks, or designing the "defense" to adversarial examples, is a challenging problem. Athalye et al. (2018); Uesato et al. (2018) showed that many proposed defenses are broken and do not significantly increase robustness under adaptive attacks. So far, state-of-the-art defense methods include adversarial training (Madry et al., 2018; Sinha et al., 2018) and optimizing a certified bound on robustness (Wong and Kolter, 2018; Raghunathan et al., 2018a; Wang et al., 2018a; Mirman et al., 2018), but there is still a long way to go to conquer the adversarial example problem. On MNIST at perturbation $\epsilon = 0.3$, adversarially trained models (Madry et al., 2018) are resistant to strong adversarial attacks but cannot be efficiently certified using existing neural network verification techniques. On the other hand, certifiable training methods usually suffer from high clean and verified error; in (Wong et al., 2018), the best model achieves 43.1% verified error and 14.9% clean error, which is much higher than ordinary MNIST models.

Most of these existing defense methods only focus on improving the robustness of a single model. Traditionally, a model ensemble has been used to improve prediction accuracy of weak models. For example, voting or bootstrap aggregating (bagging) can be used to improve prediction accuracy in many occasions. Furthermore, boosting based algorithms, including AdaBoost (Freund and Schapire, 1997), LogitBoost (Friedman et al., 2000), gradient boosting (Friedman, 2001, 2002) and many other variants, are designed to minimize an upper bound (surrogate loss) of classification error, which provably increases the model's accuracy despite the fact that each base model can be very weak. Inspired by the successful story of model ensembles, our question is that if a similar technique can be used to build a robust model with better *provable* robustness via an ensemble of certifiable base models?

Intuitively, attacking an ensemble seems to be harder than attacking a single model, because an adversary must fool all models simultaneously. Some works on using a model ensemble to defend against adversarial examples (Abbasi and Gagné, 2017; Strauss et al., 2017; Liu et al., 2018; Pang et al., 2019; Kariyappa and Qureshi, 2019) show promising results that they can indeed increase the required adversarial distortion for a successful attack and improve robustness. However, none of these works attempt to propose a *provable* (or *certifiable*) method to improve model robustness via an ensemble, so there is no guarantee that these methods work in all situations. For example, He et al. (2017) reported that attacking a specialist ensemble only increases the required adversarial distortion by as little as 6% compared to a single model.

In this paper, we propose a new algorithm, **RobBoost**, that can provably enhance the robustness certificate of

This is an extended version of ICLR 2019 Safe Machine Learning Workshop paper, "RobBoost: A provable approach to boost the robustness of deep model ensemble".
May 6, 2019. New Orleans, LA, USA

a deep model ensemble. First, we consider the setting where a set of pretrained robust models are given, and we aim to find the optimal weights for each base classifier that maximize provable robustness. We select the weight for each base model iteratively, to maximize a lower bound on the classification margin of the neural network ensemble classifier. Given a set of T base models which are individually certifiable, we formulate a certified robustness bound for the model ensemble and show that solving the optimal ensemble leads to an optimization problem. We propose a coordinate descent based algorithm to iteratively solve the RobBoost objective. Second, we consider training each base classifier sequentially from scratch, in a setting more similar to traditional gradient boosting, where each model is sequentially trained to improve the overall robustness of the current ensemble. Our experiments show that RobBoost can select a set of good base classifiers and weight them optimally, outperforming a naive average of all base models; on MNIST with ℓ_∞ perturbation of $\epsilon = 0.3$, RobBoost reduces verified error from 36% (averaging models) to 34% using 12 certifiable base models trained individually.

2 Related Work

2.1 Neural network robustness verification

The robustness of a neural network can be verified by analyzing the reachable range of an output neuron for *all* possible inputs within a set (for example, a perturbed image with bounded ℓ_∞ norm), thus the margins of predictions between the top-1 class and other classes can be examined. Unfortunately, finding the exact reachable range is NP-complete (Katz et al., 2017) and is equivalent to a mixed integer linear programming problem (Tjeng et al., 2019; Xiao et al., 2019). Therefore, many recent works in robustness verification develop computationally tractable ways to obtain outer bounds of reachable ranges. As robustness verification can be cast into a non-convex minimization problem (Ehlers, 2017), one approach is to resort to duality and give a lower bound of the solution (Dvijotham et al., 2018c; Qin et al., 2019; Dvijotham et al., 2018a). Also, we can relax the primal optimization problem with linear constraints, and obtain a lower bound of the original problem using linear programming (LP) or the dual of LP (Wong and Kolter, 2018; Wong et al., 2018). However, solving LPs for a relatively large network can still be quite slow (Salman et al., 2019). Fortunately, the relaxed LP problem can be solved greedily in the primal space (Zhang et al., 2018; Weng et al., 2018; Wang et al., 2018b) or in the dual space (Wong and Kolter, 2018). "Abstract transformers" (Singh et al., 2018, 2019a; Gehr et al., 2018; Mirman et al., 2018; Singh et al., 2019b) propagate an abstraction of input regions layer by layer to eventually give us bounds on output neurons. See (Salman et al., 2019; Liu et al., 2019) for a comprehensive discussion on the connections between these algorithms.

Our work relies on the neural network outer bounds proposed in (Zhang et al., 2018; Weng et al., 2018; Wong and Kolter, 2018), which are the state-of-the-art methods to efficiently give an upper and a lower bound for an output neuron given an ℓ_p-norm bounded input perturbation. These bounds are essentially linear with respect to input perturbations, which is crucial for developing a tractable framework (see Proposition 3.1 for more details). Besides these methods, local Lipschitz constants can also be used for efficiently giving a formal robustness guarantee (Hein and Andriushchenko, 2017; Zhang et al., 2019); tighter bounds can be obtained by semidefinite programming (SDP) based methods (Raghunathan et al., 2018a,b; Dvijotham et al., 2019), although they scale much poorly to larger models.

2.2 Defending against adversarial examples

We categorize existing defending techniques into two categories: *certified* defenses that can provably increase the robustness of a model, and *empirical* defenses that are mostly based on heuristics that have not been proven to improve robustness with a formal guarantee. Empirical defenses, even sophisticated ones, can possibly be evaded using stronger or adaptive attacks (Athalye et al., 2018; Carlini and Wagner, 2017a,c,b). We mostly focus on certified defenses in our paper, since our proposed method requires certified base models to create a certifiable ensemble.

Since many robustness verification methods give us a lower bound on the output margin between the ground-truth class and other classes under norm bounded input perturbation, optimizing a loss containing this bound obtained by a differentiable verification method will lead to maximizing this margin and improving *verified error* on the training set. For example, Wong et al. (2018); Wong and Kolter (2018) propose to minimize the verified error through a cross-entropy loss surrogate using LP relaxation based verification bounds; Wang et al. (2018a) optimize a similar verification bound; Dvijotham et al. (2018b) proposes to learn the dual variables in dual relaxations using a learner network during training; Mirman et al. (2018) propose an differentiable version of abstract transformers and include the obtained bounds in loss function to provably increase robustness. Raghunathan et al. (2018a) propose to control the global Lipschitz constant of a 2-layer network to give a certified robustness bound, but it cannot be easily extended to multiple layers.

Previous *certified* defenses mostly focus on improving

the robustness of a single model. In Wong et al. (2018), multiple models are considered in a cascaded manner, where each subsequent model is trained using the examples that cannot be certified by previous models. In inference time, only the last cascaded model is considered. RobBoost works in a different scenario, where we consider how to combine certified base classifiers to a stronger one with better verified error. Empirical defenses using ensembles include (Pang et al., 2019; Kariyappa and Qureshi, 2019; Abbasi and Gagné, 2017; Strauss et al., 2017), however they do not provide provable robustness guarantees.

3 The RobBoost Algorithm

Notations. We define a C-class H-layer feed-forward classification neural network $f(\boldsymbol{x}) \in \mathbb{R}^C$ as

$$f(\boldsymbol{x}) = \mathbf{W}^{(H)} h^{(H-1)}(\boldsymbol{x}) + \mathbf{b}^{(H)}$$
$$h^{(l)}(\boldsymbol{x}) = \sigma^{(l)}(\mathbf{W}^{(l)} h^{(l-1)}(\boldsymbol{x}) + \mathbf{b}^{(l)}), \forall l \in \{1, \dots, H-1\},$$

where $h^{(0)} := \boldsymbol{x}$ and layer l's weight matrix is $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ and bias vector is $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$. Input $\boldsymbol{x} \in \mathbb{R}^d$ and for convenience we define $n_0 = d$. $\sigma^{(l)}$ is a component-wise activation function. For convenience, we denote the row vector $\mathbf{A}_{(j,:)}$ as the j-th row of matrix $\mathbf{A}$, column vector $\mathbf{A}_{(:,i)}$ as the i-th column of matrix $\mathbf{A}$. Additionally, We define the ℓ_p ball centered at $\boldsymbol{x}_0$ with radius ϵ as $B_p(\boldsymbol{x}_0, \epsilon) := \{\boldsymbol{x} | \|\boldsymbol{x} - \boldsymbol{x}_0\|_p \le \epsilon\}$, $1 \le p \le \infty$. We use $[n]$ to denote the set $\{1, \cdots, n\}$. We denote a training example as a pair $(\boldsymbol{x}_k, y_k)$, where $\boldsymbol{x}_k \in \mathbb{R}^d$, $y_k \in [C]$ is the class label, $k \in [N]$ and N is the total number of training examples.

3.1 Linear outer bounds for neural networks

We start with guaranteed linear upper and lower bounds for a single neural network $f(\boldsymbol{x})$:

Proposition 3.1 (Linear outer bounds of neural networks)**.** *A neural network function $f(\boldsymbol{x}) \in \mathbb{R}^C$ can be linearly upper and lower bounded for all $\boldsymbol{x} = \boldsymbol{x}_k + \Delta\boldsymbol{x}$, where $\Delta\boldsymbol{x} \in B_p(0, \epsilon)$:*

$$\mathbf{L}_{(j,:)}\Delta\boldsymbol{x} + \mathbf{c}_j \le [f(\boldsymbol{x})]_j \le \mathbf{U}_{(j,:)}\Delta\boldsymbol{x} + \mathbf{d}_j, \forall j \in [C] \quad (1)$$

where $\mathbf{L}, \mathbf{U} \in \mathbb{R}^{C \times d}$ and $\mathbf{c}, \mathbf{d} \in \mathbb{R}^C$ depend on $\boldsymbol{x}_k$, p, ϵ.

Proposition 3.1 is a direct consequence of Theorem 3.2 in CROWN (Zhang et al., 2018), which gives the explicit form of $\mathbf{L}, \mathbf{U}, \mathbf{c}, \mathbf{d}$ as a function of neural network parameters. A similar outcome can be obtained from the neural network verification literature (Wong and Kolter, 2018; Singh et al., 2018; Weng et al., 2018; Wang et al., 2018b), but CROWN typically gives the tightest bound. We assume that $\mathbf{L}, \mathbf{U}, \mathbf{c}, \mathbf{d}$ are pre-computed for each example for a given ϵ using CROWN or other similar algorithms.

To verify if we can change the output of the network from the ground-truth class i to another class j, we desire to obtain a lower bound on the margin $[f(\boldsymbol{x})]_i - [f(\boldsymbol{x})]_j$. For a training example $(\boldsymbol{x}_k, y_k)$, we define the margin between the true class $i = y_k$ and other $C-1$ classes as a vector function $\widehat{f}(\boldsymbol{x}_k) \in \mathbb{R}^{C-1}$, where $[\widehat{f}(\boldsymbol{x}_k)]_j = [f(\boldsymbol{x}_k)]_i - [f(\boldsymbol{x}_k)]_j$, for $j < i$ and $[\widehat{f}(\boldsymbol{x}_k)]_j = [f(\boldsymbol{x}_k)]_i - [f(\boldsymbol{x}_k)]_{j+1}$ for $j \ge i$, $j \in [C-1]$. To obtain a lower bound of margin, we define the new network $\widehat{f}(\boldsymbol{x})$ with the same weights and biases as $f(\boldsymbol{x})$, except that the last layer H of $\widehat{f}(\boldsymbol{x})$ is reformed as:

$$\widehat{\mathbf{W}}^{(H)}_{(j,:)} = \mathbf{W}^{(H)}_{(i,:)} - \mathbf{W}^{(H)}_{(s(j,i),:)} \quad (2)$$

$$\widehat{\mathbf{b}}^{(H)}_j = \mathbf{b}^{(H)}_i - \mathbf{b}^{(H)}_{s(j,i)} \quad (3)$$

$$\text{where } j \in [C-1],\ s(j,i) = \begin{cases} j & j < i \\ j+1 & j \ge i \end{cases}$$

$s(\cdot, i)$ is a $[C-1] \to [C]$ mapping for j that skips the class i (ground truth class). Then we can lower bound the margins $\widehat{f}(\boldsymbol{x})$ by Proposition 3.1:

$$\widehat{\mathbf{L}}_{(j,:)}\Delta\boldsymbol{x} + \widehat{\mathbf{c}}_j \le [\widehat{f}(\boldsymbol{x})]_j, \forall j \in [C-1],$$

where $\widehat{\mathbf{L}} \in \mathbb{R}^{(C-1)\times d}$ and $\widehat{\mathbf{c}} \in \mathbb{R}^{C-1}$ implicitly depend on $(\boldsymbol{x}_k, y_k)$, ϵ and p. For *every* $\boldsymbol{x} = \boldsymbol{x}_k + \Delta\boldsymbol{x}$, $\Delta\boldsymbol{x} \in B_p(0, \epsilon)$, the following bound holds[1]:

$$[\widehat{f}(\boldsymbol{x})]_j \ge \widehat{\mathbf{L}}_{(j,:)}\Delta\boldsymbol{x} + \widehat{\mathbf{c}}_j \ge -\epsilon\|\widehat{\mathbf{L}}_{(j,:)}\|_q + \widehat{\mathbf{c}}_j, \quad (4)$$

where $\|\cdot\|_q$ is the dual norm of $\|\cdot\|_p$. When $[\widehat{f}(\boldsymbol{x})]_j > 0$ for *all* $\boldsymbol{x} \in B_p(\boldsymbol{x}_k, \epsilon)$, we cannot change the classification of $\boldsymbol{x}$ from class i to any other class, thus no adversarial examples exist. The q-norm of $\widehat{\mathbf{L}}_{(j,:)}$ plays an important rule in the model's robustness. We define the margin for target class $s(j, y_k)$ based on the guaranteed lower bound (4):

$$M(\boldsymbol{x}_k, j) := -\epsilon\|\widehat{\mathbf{L}}_{(j,:)}\|_q + \widehat{\mathbf{c}}_j \quad (5)$$

With this lower bounds on margin, we can extend the definition of ordinary classification error and define *verified error*, which is a provable upper bound on error under *any* norm bounded attacks:

Definition 3.2 (Robustness Certificate and Verified Error)**.** *Given a perturbation radius ϵ where an input $\boldsymbol{x}_k$ can be perturbed arbitrarily within $B_p(\boldsymbol{x}_k, \epsilon)$, verified error is the percentage of examples that do* not *have a provable robustness certificate:*

$$\text{Verified Error} := 1 - \frac{|\{k | M(\boldsymbol{x}_k, j) > 0 \text{ for all } j \in [C-1]\}|}{N} \quad (6)$$

[1] As in Zhang et al. (2018) and many other works, we illustrate unbounded perturbation (not limited to $[0, 1]$) to give the lower bound. Using bounded perturbation on MNIST typically improves verified error by 1-2%.